

KUPC Phase 4 — Resume Upload & AI Analysis

Status: Specification (Planned)
Date: 2026-07-11
Depends on: Phase 3 (3A design + 3B implementation) — complete
References: PHASE_3A_DOCUMENTATION.md, PHASE_3B_DOCUMENTATION.md, INTEGRATION.md (response shape only)
Feeds into: Phase 5 — Student & Company Profiles; later swipe / match / chat phases

Milestone	Topic	Status
1	Contracts & module scaffold	Planned
2	Schema & repository extensions	Planned
3	Supabase Storage & upload pipeline	Planned
4	Async worker infrastructure	Planned
5	PDF text extraction	Planned
6	OpenAI scoring service	Planned
7	Persistence & active-resume linkage	Planned
8	Read APIs, polling & Swagger	Planned
9	Testing matrix & hardening	Planned

What Phase 4 Is

Phase 4 turns the Phase 3 `resumes` and `resume_analysis` tables into a working product pipeline:

1. A student uploads a PDF resume.
2. The file is stored in Supabase Storage and a `resumes` row is created.
3. An analysis job is enqueued (async).
4. A worker extracts text, calls OpenAI for structured ATS-style scoring, and writes `resume_analysis`.
5. The student's active resume pointer (`students.resume_id`) is updated.

Phase 2 answered: *Who is making this request?*

Phase 3 answered: *Where do resumes and analyses live in the database?*

Phase 4 answers: *How does a resume get uploaded, scored, and persisted?*

Why this phase exists

Placement features (profiles, swipe eligibility, company review) need structured resume data: skills, ATS score, strengths, and suggestions. Calling OpenAI inside the HTTP request would:

- Block the client for several seconds
- Time out under load
- Make retries and failure handling awkward

An **upload pipeline + async worker** keeps the API fast and the analysis reliable.

Design decisions (locked for this phase)

Decision	Choice	Why
AI provider	OpenAI (structured JSON)	Explicit Phase 4 requirement; richer feedback than rule-only scoring
Response shape	Inspired by <code>INTEGRATION.md</code> / <code>analytiCV</code>	Familiar ATS score, breakdown, strengths, suggestions — not the Python spaCy runtime
Async	BullMQ on Redis	Project already has <code>REDIS_URL</code> + <code>ioredis</code> for auth cache
File storage	Supabase Storage	Matches existing Supabase stack; <code>resumes.file_url</code> already exists

Scope	Backend only	Module, APIs, worker, persistence, tests — no frontend UI this phase
-------	--------------	--

End-to-end flow

```
Student (JWT, role=STUDENT) ■ ▼ POST /api/v1/resumes (multipart PDF) ■ ■■■ Validate PDF (type, size) ■■■ Upload to Supabase Storage bucket `resumes` ■■■ INSERT public.resumes (file_url, file_name, student_id) ■■■ INSERT public.resume_analysis (status=pending) ■■■ Enqueue BullMQ job { resumeId, analysisId, studentId } ■■■ 202 Accepted { resumeId, analysisId, status: "pending" } Worker (separate process) ■ ■■■ Mark analysis status=processing ■■■ Download PDF from Storage ■■■ Extract plain text ■■■ Call OpenAI → structured score + skills + feedback ■■■ UPDATE resume_analysis (completed | failed) ■■■ UPDATE students.resume_id = resumeId (on success) Student polls GET /api/v1/resumes/:id/analysis ■■■ pending | processing | completed | failed
```

Out of scope

- Frontend analyzer / builder / editor / chat UI (analytiCV surfaces)
- Job-description fit matching (future)
- Company verification document upload (separate storage flow)
- Replacing Phase 3 schema wholesale (extend only)

Milestone 1 — Contracts & Module Scaffold

What it is

Milestone 1 locks the **API contracts**, **analysis JSON schema**, **status enum**, and the **feature module layout** before any storage or OpenAI code lands. It mirrors how Phase 2 started with `src/modules/auth/`.

Why it happens first

Without shared contracts, upload, worker, and read endpoints invent incompatible shapes. Frontend and tests need stable field names from day one. Scaffolding the module early keeps Phase 4 code out of ad-hoc route files.

How it will be completed

1.1 Status model

Analysis lifecycle:

Status	Meaning
pending	Row created; job not yet started
processing	Worker claimed the job
completed	OpenAI result persisted
failed	Terminal failure after retries (or non-retryable error)

Transitions: `pending` → `processing` → `completed` | `failed`. Never rewrite a `completed` row in place for a re-run; insert a **new** `resume_analysis` row (Phase 3A rule: many analyses per resume).

1.2 Analysis result JSON (OpenAI → API)

Align with `INTEGRATION.md` concepts:

```
{ "ats_score": { "total_score": 72, "grade": "B", "breakdown": [ { "category": "contact_info",
"label": "Contact Info", "score": 15, "max_score": 20 } ] }, "extracted_skills": { "languages":
["Python", "TypeScript"], "frameworks": ["React"], "databases": ["PostgreSQL"], "cloud": [],
"data_ml": [], "other": ["Git"] }, "summary": "Strong technical resume with clear experience
bullets.", "strengths": ["Quantified achievements", "Clear contact block"], "suggestions": [ {
"suggestion": "Add metrics to the most recent role.", "category": "experience_writing",
"priority": "high" } ], "issues_identified": ["Few action verbs in older roles"] }
```

1.3 HTTP contracts (draft)

Method	Path	Auth	Purpose
--------	------	------	---------

POST	/api/v1/resumes	Student	Upload PDF; enqueue analysis
GET	/api/v1/resumes	Student	List own resumes
GET	/api/v1/resumes/:id	Student (owner)	Resume metadata
GET	/api/v1/resumes/:id/analysis	Student (owner)	Latest analysis (+ status)
DELETE	/api/v1/resumes/:id	Student (owner)	Delete resume (+ storage object)

Upload success: **202 Accepted** with { resumeId, analysisId, status: "pending" }.

1.4 Module scaffold

Create:

```
src/modules/resumes/ resumes.constants.ts resumes.types.ts resumes.validation.ts
resumes.controller.ts resumes.service.ts resumes.routes.ts index.ts
```

Wire routes through `src/routes/index.ts` under `/api/v1/resumes`. Controllers stay thin; services own orchestration; repositories remain the only table access path (Phase 3 boundary).

1.5 Steps

1. Add TypeScript types for upload response, analysis status, and OpenAI result DTO.
2. Add Zod schemas for multipart metadata and path params.
3. Create empty controller/service/routes exporting a router.
4. Mount router behind `authenticate + authorize('STUDENT')`.
5. Document contracts in this file and prepare Swagger stubs (filled in Milestone 8).

Milestone 1 exit checklist

Item	Done when
Status enum documented and typed	pending \ processing \ completed \ failed
Analysis JSON shape documented	Matches section 1.2
HTTP routes listed	Table 1.3 agreed
Module folder exists	<code>src/modules/resumes/*</code> scaffold

Routes mounted with student auth	Unauthorized → 401; non-student → 403
----------------------------------	---------------------------------------

Milestone 2 — Schema & Repository Extensions

What it is

Milestone 2 extends Phase 3 `resume_analysis` so the async pipeline can track lifecycle and store richer OpenAI output. It also extends `resumes.repository.ts`.

Why it is needed

Phase 3 created:

- `resumes(id, student_id, file_url, file_name, uploaded_at)`
- `resume_analysis(id, resume_id, extracted_skills, ats_score, summary, analyzed_at)`

That is enough for a completed score, but **not** for pending/failed jobs, error messages, grade, breakdown, strengths, or suggestions. Without status columns, the client cannot poll meaningfully.

How it will be completed

2.1 Migration (new file)

Suggested path:

```
supabase/migrations/20260711000000_phase4_resume_analysis.sql
```

Add to `resume_analysis` (idempotent `ADD COLUMN IF NOT EXISTS`):

Column	Type	Notes
<code>status</code>	TEXT NOT NULL DEFAULT 'pending'	CHECK: pending, processing, completed, failed
<code>error_message</code>	TEXT	Set when failed
<code>grade</code>	TEXT	e.g. A/B/C/D
<code>score_breakdown</code>	JSONB	Category scores
<code>strengths</code>	JSONB	Array of strings
<code>suggestions</code>	JSONB	Array of suggestion objects
<code>issues_identified</code>	JSONB	Array of strings

<code>raw_response</code>	JSONB	Optional full OpenAI payload for debug
<code>model</code>	TEXT	Model id used
<code>started_at</code>	TIMESTAMPTZ	When processing began
<code>completed_at</code>	TIMESTAMPTZ	When finished

Keep existing `extracted_skills`, `ats_score`, `summary`, `analyzed_at`.

Optional (if skill containment queries are needed):

```
CREATE INDEX IF NOT EXISTS idx_resume_analysis_skills ON public.resume_analysis USING gin
(extracted_skills);
```

Register the migration in `scripts/apply-migrations.mjs`.

2.2 Repository extensions

Extend `src/database/resumes.repository.ts`:

- `createAnalysis` — accept status: `'pending'`
- `updateAnalysisStatus` — `pending` → `processing`
- `completeAnalysis` — write scores + JSON fields + `completed`
- `failAnalysis` — write `error_message` + `failed`
- `findAnalysisById` / keep `findAnalysisByResumeId` (latest by `analyzed_at` or `completed_at`)

Do not add `.from('resumes')` outside `src/database/` (Phase 3 boundary test must still pass).

2.3 Steps

1. Write migration SQL with CHECKs and defaults.
2. Apply via `npm run db:migrate`.
3. Extend repository types and methods.
4. Add static schema test assertions for new columns (Phase 4 test file in Milestone 9).
5. Update this doc's schema section if columns change during implementation.

Milestone 2 exit checklist

Item	Done when
------	-----------

Migration applied	New columns exist in Supabase
Status CHECK enforced	Invalid status → Postgres error
Repository methods cover pending/complete/fail	Typed + throw on error
Boundary rule preserved	No table access outside repositories
apply-migrations.mjs lists new file	Ordered after Phase 3 migrations

Milestone 3 — Supabase Storage & Upload Pipeline

What it is

Milestone 3 implements the **synchronous half** of the pipeline: accept a PDF, store it, create DB rows, enqueue analysis (enqueue wiring finalized in Milestone 4).

Why it matters

`resumes.file_url` is useless without a real object store. Phase 2 deliberately deferred file upload; Phase 4 owns student resume storage. Validation here prevents garbage files from reaching OpenAI.

How it will be completed

3.1 Storage bucket

- Bucket name: `resumes` (private)
- Object key pattern: `{studentId}/{resumeId}/{originalFileName}`
- Access: service role for upload/download in API/worker; students never get the service key
- Signed URLs for download if the client needs a temporary link (optional in this phase)

3.2 Upload validation

Rule	Value
MIME	<code>application/pdf</code> only
Extension	<code>.pdf</code>
Max size	5 MB (configurable via env)
Auth	JWT + role <code>STUDENT</code>
Ownership	<code>student_id = req.user.id</code>

Reject non-PDF and oversized files with stable error codes (e.g. `RESUME_INVALID_TYPE`, `RESUME_TOO_LARGE`).

3.3 Upload sequence (service)

1. Parse multipart (`multer` memory or temp disk).
2. Validate type/size.
3. Generate `resumeId` (UUID) client-side of insert **or** insert then use returned id for storage path — prefer: create UUID, upload with that id in path, then insert row with same id if repository supports it; otherwise upload after

insert using returned id.

4. Upload bytes to Supabase Storage.
5. `resumesRepository.create({ studentId, fileUrl, fileName })`.
6. `resumesRepository.createAnalysis({ resumeId, status: 'pending' })`.
7. Enqueue job (Milestone 4).
8. Return **202** with ids + status.

On storage failure: do not leave orphan DB rows (compensate: delete row or fail before insert). Prefer **upload then insert**, or transactional cleanup in `catch`.

3.4 Env / config

Variable	Purpose
<code>RESUME_MAX_BYTES</code>	Max upload size (default 5242880)
<code>RESUME_STORAGE_BUCKET</code>	Default <code>resumes</code>
Existing Supabase keys	Service role for storage

3.5 Steps

1. Create Storage bucket in Supabase (dashboard or SQL/storage API).
2. Add `multer` (or equivalent) dependency.
3. Implement `resumes.service.uploadResume`.
4. Implement `POST /api/v1/resumes` controller.
5. Add rate limit on upload route (stricter than general API).
6. Manual smoke: upload PDF as student → row + object exist.

Milestone 3 exit checklist

Item	Done when
Bucket exists and is private	Confirmed in Supabase
PDF-only + size limit enforced	Non-PDF / huge file rejected
<code>resumes</code> row created with real <code>file_url</code>	DB + Storage aligned

Pending resume_analysis created	Status pending
Non-students blocked	403
Orphan handling documented/implemented	Failed upload does not leave junk

Milestone 4 — Async Worker Infrastructure

What it is

Milestone 4 adds **BullMQ** on Redis so analysis runs outside the HTTP request. A dedicated worker process consumes jobs.

Why async

OpenAI + PDF extract can take 5–30+ seconds. HTTP should return immediately. Retries, backoff, and concurrency control belong in a queue, not in Express middleware.

How it will be completed

4.1 Dependencies & process

- Add `bullmq` (uses Redis; project already has `ioredis` / `REDIS_URL`).
- Worker entry: `src/workers/resumeAnalysis.worker.ts`
- `npm` script: `"worker:resumes": "ts-node-dev --respawn --transpile-only src/workers/resumeAnalysis.worker.ts"`
- Require `REDIS_URL` for upload enqueue + worker (fail fast if missing in production).

4.2 Queue design

Item	Value
Queue name	<code>resume-analysis</code>
Job name	<code>analyze</code>
Payload	<code>{ resumeId, analysisId, studentId }</code>
Attempts	3
Backoff	Exponential (e.g. 5s, 15s, 45s)
Concurrency	2–5 (config)
Idempotency	If analysis already <code>completed</code> , no-op success

4.3 Worker responsibilities (orchestration only)

1. Load analysis + resume via repositories.

2. Set status `processing` + `started_at`.
3. Call `extract` (M5) → `OpenAI` (M6) → `persist` (M7).
4. On final failure: `failAnalysis` with message; do not leave `processing` forever.

4.4 Steps

1. Add BullMQ dependency and shared Redis connection helper (reuse URL from env).
2. Implement `enqueueResumeAnalysis(payload)` used by upload service.
3. Implement worker processor stub (log job; complete no-op until M5–M7).
4. Document runbooks: API process + worker process both required in deploy.
5. Add graceful shutdown (close queue/worker on SIGTERM).

Milestone 4 exit checklist

Item	Done when
Job enqueued on successful upload	Visible in Redis/BullMQ
Worker process starts and consumes	Logs show job received
Retries configured	Transient failures re-attempt
Missing Redis fails clearly	Error message / health note
Idempotent completed jobs	Re-delivery safe

Milestone 5 — PDF Text Extraction

What it is

Milestone 5 converts the stored PDF into **plain text** suitable for an OpenAI prompt.

Why it is a separate milestone

Extraction failures (scanned image-only PDFs, corrupt files, empty text) should be classified **before** spending OpenAI tokens. Keeping extract as its own service also allows swapping libraries later.

How it will be completed

5.1 Library choice

Default: pdf-parse (Node) for text-layer PDFs. Document limitation: image-only scans may yield empty text → mark analysis `failed` with `RESUME_EMPTY_TEXT` (OCR is out of scope for Phase 4).

5.2 Service

```
src/modules/resumes/resumes.extract.ts extractTextFromPdf(buffer: Buffer): Promise<string>
```

Worker flow:

- 1. Download object from Storage using `file_url` / storage path.
- 2. Extract text.
- 3. Trim; if length below threshold (e.g. < 100 chars), fail analysis.
- 4. Optional light heuristic: require resume-like keywords (experience, education, skills) — soft check; log warning or fail if clearly not a resume.

5.3 Steps

- 1. Add PDF dependency.
- 2. Implement extract helper with unit tests on fixture PDFs.
- 3. Integrate into worker before OpenAI call.
- 4. Map empty/corrupt PDF to non-retryable failure where appropriate.

Milestone 5 exit checklist

Item	Done when
------	-----------

Text extracted from normal PDF	Fixture test passes
Empty/corrupt handled	Analysis <code>failed</code> with clear code
Worker downloads from Storage	Integration path works
OCR explicitly out of scope	Documented

Milestone 6 — OpenAI Scoring Service

What it is

Milestone 6 calls OpenAI with the extracted resume text and returns **structured** ATS-style scoring and feedback.

Why OpenAI here

Phase 4 requires AI scoring. Structured outputs keep persistence deterministic (numeric `ats_score`, typed JSON for skills/suggestions) instead of free-form prose that is hard to store and query.

How it will be completed

6.1 Config

Variable	Purpose
OPENAI_API_KEY	Required for worker
OPENAI_MODEL	Default documented model (e.g. gpt-4o-mini)
OPENAI_TIMEOUT_MS	Request timeout

Never commit keys. Validate in `env.ts`.

6.2 Service

```
src/modules/resumes/resumes.openai.ts analyzeResumeText(text: string):  
Promise<ResumeAnalysisResult>
```

- Use OpenAI SDK with **JSON schema / response_format** matching Milestone 1.2.
- System prompt: score ATS readiness (contact, skills, experience, education, formatting); return strengths, suggestions, issues; extract skills into taxonomy categories.
- Temperature low (e.g. 0–0.3) for stability.
- Store `model` used on the analysis row.

6.3 Scoring guidance (prompt contract)

Ask the model for `total_score` 0–100 and category breakdowns similar to analytiCV weights (contact / skills / experience / education / formatting) so UI can show a familiar breakdown. Grades: A ≥ 85, B ≥ 70, C ≥ 55, else D (compute server-side from total if model omits grade).

6.4 Error handling

Error	Behavior
Rate limit / 5xx	Retry via BullMQ
Invalid JSON / schema mismatch	Retry once; then fail
Auth / bad API key	Fail fast; alert ops (non-retryable)

6.5 Steps

1. Add `openai` package.
2. Implement schema-validated `analyze` function.
3. Unit-test with mocked OpenAI client.
4. Wire into worker after extraction.
5. Log token usage optionally via `analytics_events` (optional stretch).

Milestone 6 exit checklist

Item	Done when
Structured result matches contract	Zod/parse validates
Env vars documented	<code>.env.example</code> updated
Mocked unit tests pass	No live key required in CI
Retries vs fail-fast classified	Documented in code comments + this doc
Model id persisted	<code>resume_analysis.model</code> set

Milestone 7 — Persistence & Active-Resume Linkage

What it is

Milestone 7 writes the OpenAI result into `resume_analysis` and updates `students.resume_id` to the newly uploaded resume.

Why

Phase 3A defined the product rule:

```
Upload → INSERT resumes → Analysis job → INSERT/UPDATE resume_analysis → UPDATE
students.resume_id Old resumes + analyses remain as history.
```

Profiles and later phases read the **active** resume via `students.resume_id`.

How it will be completed

7.1 On success

1. `completeAnalysis` with `scores`, `skills`, `summary`, `strengths`, `suggestions`, `issues`, `breakdown`, `status=completed`, `completed_at=now()`.
2. `studentsRepository` method to `setActiveResume(studentId, resumeId)`.
3. Optionally upsert `student_skills` from `extracted_skills` flattened names (match/create rows in `skills`). Treat as best-effort; do not fail analysis if skill sync fails — log and continue.

7.2 On failure

1. `failAnalysis(analysisId, errorMessage)`.
2. Do **not** change `students.resume_id` if this upload never completed successfully (keep previous active resume).
3. If product wants “latest upload even if failed” as active file pointer, document that explicitly — **default: only promote on completed analysis**.

7.3 Steps

1. Extend `students repository` with `setActiveResume`.
2. Implement `complete/fail` paths in worker.
3. Add transaction-like ordering: persist analysis first, then set active resume.
4. Verify `CASCADE/SET NULL` behavior still holds (Phase 3 tests).

Milestone 7 exit checklist

Item	Done when
Completed analysis row fully populated	All key columns set
<code>students.resume_id</code> updated on success	Points to new resume
Failed analysis does not clobber active resume	Default rule verified
History preserved	Prior resumes/analyses remain
Optional skill sync documented	Behavior clear if enabled

Milestone 8 — Read APIs, Polling & Swagger

What it is

Milestone 8 exposes read/delete endpoints and documents them in Swagger so clients can poll analysis status and display results.

Why

Upload returns 202; the UI must poll until `completed` or `failed`. Without read APIs, the pipeline is not usable.

How it will be completed

8.1 Endpoints

Method	Behavior
GET <code>/api/v1/resumes</code>	List current student's resumes (newest first)
GET <code>/api/v1/resumes/:id</code>	Metadata; 404 if not owner
GET <code>/api/v1/resumes/:id/analysis</code>	Latest analysis including status; 404 if none
DELETE <code>/api/v1/resumes/:id</code>	Delete Storage object + DB row (CASCADE analysis); clear active pointer via SET NULL

Ownership: always filter by `student_id = req.user.id` in the service layer (defense in depth beyond RLS).

8.2 Polling contract

Client polls GET `.../analysis` every 2–3s until status is terminal. Response includes `status`, and when completed: score fields + feedback JSON.

8.3 Swagger

Extend `src/config/swagger.ts` / JSDoc on routes with schemas for upload 202, analysis payload, and error codes.

8.4 Steps

1. Implement list/get/analysis/delete in service + controller.
2. Add ownership checks and consistent `AppError` codes.
3. Document in Swagger UI.
4. Smoke-test poll loop against worker.

Milestone 8 exit checklist

Item	Done when
All four read/delete routes work	Manual + automated tests
Non-owner cannot read	404/403
Swagger shows Phase 4 routes	Visible at <code>/api/docs</code> (or project path)
Polling returns status transitions	pending → processing → completed

Milestone 9 — Testing Matrix & Hardening

What it is

Milestone 9 is the quality gate: prove upload, queue, extraction, OpenAI mapping, persistence, authz, and failure paths.

Why

Phase 3 used a testing matrix before declaring the phase done. Phase 4 touches money-costing APIs (OpenAI) and private files — regressions are expensive.

How it will be completed

9.1 Test files (suggested)

File	Kind	Covers
<code>phase4.schema.test.ts</code>	Static	Migration columns / CHECKs
<code>phase4.upload.test.ts</code>	HTTP	Auth, validation, 202
<code>phase4.worker.test.ts</code>	Unit	Status transitions, idempotency
<code>phase4.openai.test.ts</code>	Unit	Schema parse with mock
<code>phase4.extract.test.ts</code>	Unit	Fixture PDFs
<code>phase4.matrix.test.ts</code>	Live (optional)	Storage + DB with env

Script: `npm run test:phase4.`

9.2 Hardening

- Upload rate limit
- Max concurrent analyses per student (optional)
- Redact raw resume text from logs
- Ensure `OPENAI_API_KEY` never logged
- Worker health / failed-job metrics (basic logging)

9.3 Steps

1. Add Jest suites with mocks for Storage/OpenAI/Redis where needed.

2. Run full suite in CI.
3. Fill Phase exit checklist evidence column.
4. Mark milestones Complete in this document when done.

Milestone 9 exit checklist

Item	Done when
Upload validation tests	Pass
Authz ownership tests	Pass
Worker success + fail paths	Pass
OpenAI mock schema tests	Pass
<code>npm run test:phase4</code> green	CI-ready
Secrets not in repo	Verified

Phase 4 Exit Checklist

#	Checklist item	Status	Evidence
1	All milestones 1–9 complete and tested	Pending	
2	Student can upload PDF and receive 202 + analysis id	Pending	
3	Worker completes OpenAI scoring and persists <code>resume_analysis</code>	Pending	
4	Failed jobs mark failed with reason; retries documented	Pending	
5	<code>students.resume_id</code> updated only on successful analysis	Pending	
6	Only owning student can access own resumes/analysis	Pending	
7	Secrets via env only (<code>OPENAI_API_KEY</code> , Redis, Supabase)	Pending	
8	Phase 4 test suite green	Pending	
9	Docs match implemented behavior	Pending	
10	API + worker both documented for deploy	Pending	

Phase 4 verdict

Not started. Implementation may begin only after this specification is accepted. Phase 3 exit checklist remains the hard dependency gate (already complete).

Appendix A — Key existing files

Path	Role
supabase/migrations/20260710000000_phase3_schema.sql	Creates resumes, resume_analysis
src/database/resumes.repository.ts	Resume + analysis repository stubs
src/middleware/authenticate.ts / authorize.ts	Auth gates
src/config/env.ts	Env validation (REDIS_URL already optional)
src/modules/auth/	Pattern for feature modules
documentation/INTEGRATION.md	analytiCV analyzer response shape reference

Appendix B — Proposed new files

Path	Role
supabase/migrations/20260711000000_phase4_resume_status_payload	Analysis payload columns
src/modules/resumes/*	Feature module
src/workers/resumeAnalysis.worker.ts	BullMQ worker
src/__tests__/phase4.*.test.ts	Phase 4 tests

Appendix C — Environment variables (Phase 4)

Variable	Required for	Notes
REDIS_URL	Upload enqueue + worker	Required in environments running analysis
OPENAI_API_KEY	Worker	Never commit
OPENAI_MODEL	Worker	Default e.g. gpt-4o-mini
OPENAI_TIMEOUT_MS	Worker	Optional
RESUME_STORAGE_BUCKET	API + worker	Default resumes
RESUME_MAX_BYTES	API	Default 5 MB
Existing SUPABASE_*	API + worker	Service role for Storage + DB

KUPC — Phase 4 specification. Implementation begins after acceptance.